

Unauthorised Access to Development Environments response

Intrusion · **High** severity · SOC IR Playbook

Incident type	Access Control Violation / Insider Threat
Severity	High
Priority	Critical
Detection sources	SIEM, Git logs, IAM, DevOps tools (e.g., Jenkins, GitLab), Audit trails, Source code version control systems

Scenario

An individual gains access to a development environment (e.g., Git repositories, staging servers, CI/CD platforms like Jenkins or GitLab CI or test databases) without authorisation. This may result in code theft, insertion of malicious code or exposure of credentials and secrets.

MITRE ATT&CK

T1059, T1087.001, T1565.002, T1606

Preparation

Enforce role-based access control Use IAM policies, SSO and least privilege for all dev tools Enable auditing and logs Track access to Git repositories, CI/CD systems and test environments Integrate logging into SIEM Stream logs from GitHub/GitLab, Jenkins, etc. Apply secret scanning tools Detect hardcoded credentials and tokens in code Conduct DevSecOps training Educate developers on secure coding and repository hygiene

Detection & Analysis

Alert from IAM/SIEM Suspicious login, token use or repo access from unusual IP or user Review Git/CI logs Check recent commits, merges, pipeline executions and user access Identify access method Direct login, API token, SSH key or OAuth integration Assess data touched Determine whether source code, pipeline configs or secrets were accessed

Containment

Revoke user/API access Disable user accounts or tokens used for access Suspend pipeline execution Pause CI/CD activities to prevent further compromise Isolate affected systems Temporarily block access to critical environments or servers Notify DevOps and security teams Coordinate containment and code review activities

Eradication

Remove unauthorised code or scripts Revert malicious commits, rollback pipeline changes Rotate secrets and credentials Especially if found in code, environment variables or configuration files Clean up compromised accounts Remove old users, service accounts or tokens Patch tool vulnerabilities Apply updates to exposed or misconfigured DevOps platforms

Recovery

Restore secure state Confirm code and pipelines are clean, access is limited to authorised users Resume CI/CD operations Only after validation of system integrity Monitor codebase and build process Set up enhanced logging and monitoring post-incident Revalidate audit controls Ensure access logs, versioning and change tracking are enabled and working

Lessons Learned & Reporting

Conduct a full review Understand attack vector, user involved and data affected Update access policies Apply tighter controls to sensitive repos and build systems Train DevOps personnel Reinforce secure access

management and code review policies Report to stakeholders Legal, compliance and clients if proprietary or regulated data is involved Document playbook updates Improve future detection and response processes in the SOC and DevOps teams

Tools typically involved

SIEM (e.g., Sentinel, Splunk); Git platforms (e.g., GitHub, GitLab, Bitbucket); CI/CD tools (e.g., Jenkins, GitLab CI, CircleCI, Azure DevOps); IAM & SSO (e.g., Okta, Azure AD, Google Workspace); Secret scanners (e.g., TruffleHog, Gitleaks); Container security tools (e.g., Aqua, Prisma Cloud)

Success metrics (SLA targets)

Detection Time <10 minutes from unauthorised access Access Revocation Time <30 minutes from alert Secret Rotation Time <1 hour for high-value tokens or keys Codebase Validation Time <24 hours Post-Incident Audit Completion Within 3 business days